

JavaScript Array Traversal using **while** Loop

The **while** loop is used to traverse an array when you want to repeat a block of code as long as a condition remains **true** .

Unlike the **for** loop, the initialization and update are written separately.

Table of Contents

- What is a while Loop?
- Why Use a while Loop?
- Syntax
- How it Works
- Examples
- Real Project Examples
- Best Practices
- Common Mistakes
- Interview Questions
- Summary Table

What is a while Loop?

Definition

A **while** loop repeatedly executes a block of code while the specified condition is **true** .

Why Use a while Loop?

We use a **while** loop to:

- Traverse arrays
- Process data until a condition becomes false
- Handle dynamic iterations
- Read unknown-length data

Syntax

```
let i = 0;

while(i < array.length){

    console.log(array[i]);

    i++;

}
```

Example 1 (Print Array)

```
const fruits = [

    "Apple",

    "Mango",

    "Orange"

];

let i = 0;

while(i < fruits.length){

    console.log(fruits[i]);

    i++;

}
```

Output

```
Apple
Mango
```

Example 2 (Sum of Numbers)

```
const numbers = [  
    10,  
    20,  
    30,  
    40  
];  
  
let i = 0;  
let sum = 0;  
while(i < numbers.length){  
    sum += numbers[i];  
    i++;  
}  
  
console.log(sum);
```

Output

```
100
```

Example 3 (Reverse Traversal)

```
const colors = [  
  "Red",  
  "Green",  
  "Blue"  
];  
  
let i = colors.length - 1;  
  
while(i >= 0){  
  console.log(colors[i]);  
  i--;  
}
```

Output

```
Blue  
Green  
Red
```

Real Project Example

Shopping Cart

```
const cart = [  
  "Laptop",  
  "Mouse",  
  "Keyboard"
```

```
];  
  
let i = 0;  
  
while(i < cart.length){  
    console.log(cart[i]);  
  
    i++;  
  
}
```

Best Practices

- Always update the loop variable.
- Use `array.length` instead of fixed numbers.
- Keep the condition simple.

Common Mistakes

Forgetting `i++`

Wrong

```
while(i < array.length){  
    console.log(array[i]);  
  
}
```

This creates an **infinite loop**.

Interview Questions

When is a `while` loop useful?

When the number of iterations is not known beforehand.

Does a `while` loop check the condition first?

Yes.

Summary Table

Feature	Description
Loop	<code>while</code>
Checks Condition	Before execution
Runs At Least Once	No
Best Use	Unknown number of iterations

Final Notes

The `while` loop is useful when the number of iterations is dynamic. For array traversal, `for` is usually preferred because it is shorter and easier to read.

JavaScript Array Traversal using `do...while` Loop

The `do...while` loop is similar to the `while` loop, but it executes the code **at least once** before checking the condition.

Table of Contents

- What is do...while?
- Why Use do...while?
- Syntax
- How it Works
- Examples
- Real Project Examples
- Best Practices
- Common Mistakes
- Interview Questions
- Summary Table

What is do...while?

Definition

A **do...while** loop executes the code first and then checks the condition.

Even if the condition is **false**, the loop runs **one time**.

Why Use do...while?

We use a **do...while** loop when:

- Code must execute at least once.
- User input needs validation.
- Menu-driven programs.
- Retry operations.

Syntax

```
let i = 0;

do{

    console.log(array[i]);

    i++;

}while(i < array.length);
```

Example 1 (Print Array)

```
const fruits = [

    "Apple",

    "Mango",

    "Orange"
```

```
];  
  
let i = 0;  
  
do{  
  
    console.log(fruits[i]);  
  
    i++;  
  
}while(i < fruits.length);
```

Output

```
Apple  
Mango  
Orange
```

Example 2 (Sum of Numbers)

```
const numbers = [  
  
    10,  
  
    20,  
  
    30  
  
];  
  
let i = 0;  
  
let sum = 0;  
  
do{  
  
    sum += numbers[i];  
  
    i++;  
  
}
```



```
}while(i < numbers.length);  
  
console.log(sum);
```

Output

```
60
```

Example 3 (Runs At Least Once)

```
const numbers = [];  
  
let i = 0;  
  
do{  
  
    console.log("Loop Executed");  
  
}while(i < numbers.length);
```

Output

```
Loop Executed
```

Real Project Example

Menu Items

```
const menu = [  
  
    "Home",
```

```
    "About",  
    "Contact"  
];  
  
let i = 0;  
  
do{  
    console.log(menu[i]);  
    i++;  
}while(i < menu.length);
```

Best Practices

- Use `do...while` only when the loop must run at least once.
- Always update the loop variable.
- Use `array.length` for the stopping condition.

Common Mistakes

Forgetting `i++`

Wrong

```
do{  
    console.log(array[i]);  
}while(i < array.length);
```

This creates an **infinite loop**.

Using `do...while` unnecessarily

If the loop does **not** need to execute at least once, prefer a **for** or **while** loop.

Interview Questions

What is the difference between **while and **do...while** ?**

while checks the condition first.

do...while executes the code first and then checks the condition.

Does **do...while always execute once?**

Yes.

Summary Table

Feature	Description
Loop	do...while
Checks Condition	After execution
Runs At Least Once	Yes
Best Use	Menus, Validation, Retry Logic

Final Notes

The **do...while** loop is less common for array traversal but is very useful when you need the loop body to execute at least once. It is commonly used in login systems, menu-driven programs, and input validation.